Source: docs\COMPLETION.md | Export: 11 November 2026

COD completion work

Local implementation only. No production mutations, deployments, rule publication, commits or external business notifications are authorized.

Ownership

The checkout starts at `7b5e54e` with a clean worktree. Shipping is already committed separately. Preserve `server/shipping.ts`, its settings UI and `quoteShipping` policy; integrate through the existing transaction and integer-paise interface. The strict free-shipping threshold uses the **pre-coupon merchandise subtotal**. Coupons reduce merchandise payable, not the qualification base. Existing login appearance, authentication, category/product and shopping/address behavior remain owned by their current modules.

Checklist

- [x] Audit architecture, existing routes, API authorization and shipping boundary.
- [x] Atomic COD placement, inventory, idempotency and recovery.
- [x] Customer orders and profile editing.
- [x] Admin fulfilment, independent COD collection and audit history.
- [x] Cancellation and policy-gated return requests.
- [x] Admin-only event notifications and per-admin reads.
- [x] Coupons, content, customers, inventory adjustments and reporting.
- [x] Storefront/SEO, security and six-width regression checks.
- [x] Packaged runtime, full tests and release/acceptance documentation.

Boundaries

COD is the only checkout method. Online payment and Razorpay remain on hold. Customer business updates appear on website order pages only; existing Firebase password recovery remains available. Returns/exchanges require explicitly approved enabled policy content; no eligibility period or refund promise is assumed.

Implementation and data

The existing single `/api/catalog` function authenticates every POST using revoked-token checking and the explicit `parinaya-silks` audience. Customer actions derive UID/email from that identity. Administrator actions require `admin: true` **and** enabled `adminAccess/{uid}`; every administrator write rereads the access record in its transaction. No frontend exception, role field, browser business write or production fixture is introduced.

Repository	Purpose / access
`orders/{sha256(uid, operationId)[0:32]}`	Trusted COD order; API-only customer projection; existing admin direct reads preserved
`customerShopping/{uid}/operations/{operationId}`	Atomic placement receipt and request hash; never a customer-selected UID
`orderOperations/{hash(uid,operationId)}`	Idempotent fulfilment/request/collection receipt
`orderSubmissionLimits/{uid_UTCday}`	At most 50 accepted COD orders per customer per UTC day
`inventoryAudit/{event}`	Before/after inventory, actor, reason, order when applicable, server timestamp
`adminNotifications/{event}`	Deduplicated trusted event; no customer feed
`adminNotificationReads/{uid}` and `/items/{event}`	Per-admin mark-all cutoff / individual reads
`coupons/{NORMALIZED_CODE}` and `/customers/{uid}`	Config/revision and atomic global/per-customer usage
`siteSettings/storeContent`	Private drafts, verified banners, featured references, approved contact/policy configuration

Repository	Purpose / access
`siteSettings/returns`	Private, approval-gated request policy derived atomically from content Save
`homeBannerMedia/{mediaId}`	Purpose/device-bound signed-upload tickets and verified metadata
Existing profiles, Bag, Wishlist, addresses and shipping	Preserved; customer profile edits are now a trusted UID-derived API action

An order keeps immutable item/name/slug/SKU/variant/image/quantity/unit-price/line-price snapshots, owned address and revision, currency, merchandise subtotal, coupon amount/configuration, shipping revision and amount, final COD total, calculation-policy version and creation time. Mutable fulfilment/collection/request/refund fields have revisions and action history. Internal notes, actor identities, collection references and external refund references are removed from customer order projections. Public content omits draft policies, administrator metadata and trusted media registry IDs.

New modules: `server/orders.ts`, `business.ts`, `content.ts`, `browse.ts`, `seo.ts`; `src/orders/*` and `src/content/*`. Integration touches the existing catalog/handler/delivery/image service, account/checkout/admin/router/catalog views, local rules/indexes and tests. Shipping calculation/settings modules are not replaced. The login image service is parameterized with a fixed separate homepage purpose/registry; product upload checks remain unchanged.

COD transaction and recovery

`orders:review` builds the same checkout read transaction as the existing address/Bag/catalog/shipping review, then checks the selected coupon. `orders:place` accepts **only** action, address ID, coupon code, review fingerprint and operation ID. It independently rebuilds that review in the same Firestore transaction that creates the order, decrements each current stock source, advances product revisions, consumes coupon usage, removes exactly the purchased Bag lines, records the operation and emits events. Every read precedes writes. Failed transactions have no partial business changes.

A changed review is returned as `{changed:true, review}` and **does not create an order**. The UI displays it and requires a new confirmation. Unsupported totals, payment status, UID or item fields are rejected. Missing/invalid shipping configuration blocks placement. Money is integer paise; BigInt performs checkout calculations. Accepted COD totals are bounded to 1,000,000,000,000 paise. Orders start `paymentStatus: due`.

The browser persists the pending operation before sending it. An uncertain retry reuses the same request/ID, including after a refresh. `orders:recover` derives the same order ID from authenticated UID and operation ID. Receipts are checked before inspecting the now-cleared Bag, so response loss cannot create a second order or delete a later Bag addition. If site storage is unavailable, placement fails before sending. My orders provides independent recovery. Pending operation storage contains references/fingerprint, never tokens or address content.

Concurrent stock reads/writes serialize in Firestore. There is no unpaid reservation, expiry or payment callback. Stock is deducted **once on accepted COD creation**. Cancellation/rejection before dispatch restores it once in the status transaction. The original variant is required; a deleted product/changed inventory structure/overflow blocks the whole operation for owner reconciliation instead of silently losing stock. Product editor saves, order stock effects and reasoned adjustments advance product revisions, preventing stale editors from overwriting stock. Audit records include before/after quantities. Low-stock events fire only on a source crossing its threshold or becoming unavailable, not on listing/page loads.

Lifecycle, requests and money

```
Placed -> Confirmed -> Processing -> Shipped -> Delivered
   |          |             |
   +----------+-------------+-> Cancelled / Rejected
```

No skipped/backwards transitions. Dispatch requires courier and tracking number; optional URLs must be HTTPS without embedded credentials. Cancellation/rejection requires a reason and cannot follow dispatch. Customer cancellation is a request; approval performs the real cancellation/restock. Repeated requests/decisions/restock operations are receipt-protected. All mutations require an expected revision; conflicts require reload.

Delivered remains COD **due**. A separate administrator collection action is permitted once after delivery and requires an external reference; actor/time/notes are recorded. Delivery never implies receipt of money.

Return/exchange requests are unavailable by default. An administrator must enter approved policy text, confirm publication approval, explicitly enable requests and select allowed request types. Requests are review-based; no automatic period, fee or eligibility promise exists. An approved return/exchange can be resolved without restock, or explicitly received/inspected/restocked once. Only a resolved request with collected COD funds supports a manual external refund record. The amount cannot exceed collections after prior refund records. These records **do not transfer money**. Uncollected cancellation is not a refund. Razorpay refunds/payments remain on hold.

Coupons, shipping and administration

Codes normalize to unique uppercase document IDs. Coupons support fixed paise or integer percentages, enablement, optional start/end timestamps (end exclusive), minimum pre-coupon merchandise subtotal, percentage cap, global and per-customer limits. Discount is floored in paise and capped at eligible merchandise. Usage increments in the order transaction only. **Cancellation does not release/recycle coupon usage**, preventing repeated

cancellation from resetting limits. Eligibility/config changes invalidate confirmation or block the coupon. Shipping uses the existing **strictly above** free threshold on the **pre-coupon** merchandise subtotal. Final total is merchandise minus coupon plus the saved shipping quote.

Admin order lists use 25-document cursor pages with server reference/exact-email/customer UID/status/COD/date filters. Export is explicitly the currently authorized filtered page; spreadsheet formula cells are escaped. Customer directory pages use 25 profiles and case-sensitive name-prefix search, including incomplete legacy profiles, with linked order history. No password hashes, account deletion, impersonation or privilege editor exist.

Dashboard aggregates query actual orders and distinguish all order value (including cancelled/rejected), delivered value, due COD on active orders, recorded external collections and recorded external refunds. Date filters refer to **order creation**, not collection cashflow dates. Loading/error is separate from zero. Low-stock pages scan at most 200 current products and expose continuation instead of presenting a partial scan as a global count. Inventory detail shows 25-entry audit pages.

Notification pages contain 50 events with continuation, exact unread count and per-admin read state. New order/request/stock alerts are written only by trusted transactions using stable event IDs. Mark-all uses a server timestamp cutoff, preserving later events as unread. The bell refreshes on open and every 60 seconds. No business email/SMS/WhatsApp/push is sent; existing user-initiated contact links and Firebase password-reset emails are retained.

Content and storefront

Store Settings preserves Login Page Appearance and Shipping Settings. Homepage banners, Website Content and Contact Information now have separate navigation sections sharing the same revision-controlled content editor/document; see CATALOG_RECOVERY.md. Up to six ordered/visible banners have independent verified desktop/mobile images, alt text, focal points, title and safe destination. Recommended desktop 1920×720 and mobile 900×900; breakpoint 768px. File upload never publishes settings, and old media is never deleted. Save/Cancel and conflict/retry retain draft semantics. Only approved policy text is public. Blank contact values retain the previously approved contact fallback. Content refreshes within 60 seconds while visible or on refresh/revisit/focus. Authentication is independent of content loading failures.

Featured product/category IDs are validated against current publication/category visibility. Public catalog always checks current enabled categories. Search/colour/stock/price/sale filters use bounded server pages and current product facts. Substring/colour filters scan at most 200 candidate records per request; an empty matching page may still have a continuation. Sorting occurs server-side before pagination. No sample promotion or review is added.

SEO uses route titles/descriptions/canonicals, private-route noindex, robots exclusions, truthful current-product JSON-LD and the existing API-backed `/sitemap.xml` rewrite. Sitemap is bounded to the first 1,000 published products and 1,000 enabled categories; larger catalogs require sitemap sharding before claiming full coverage. Metadata is client-rendered; no SSR/search-engine indexing result is claimed. The later GA4 page-view implementation is production-domain gated and requires explicit visitor opt-in; see ANALYTICS.md for environment/property settings and the remaining live verification. Historical validation results below do not establish GA4 deployment or incoming property data.

Local acceptance / future release

Start `npm.cmd run dev` and `npm.cmd run dev:api` for configured local development; automated business mutations must use `npm.cmd run test:admin` and its loopback Auth/Firestore fixtures. Never point test fixture scripts at production. `shipping:seed-local` remains emulator-only; checkout never creates missing shipping configuration.

Acceptance: sign in as emulator customer, add simple/colour items, save/select an address, review COD totals/coupon and submit. Retry a lost response/refresh and check a single order/deduction. Open My orders, then administer confirmation/processing/dispatch/delivery. Verify COD is still due until explicitly recorded collected. Test cancellation approval/restock, request policy gating, external refund recording and admin-specific notifications. Test settings Save/Cancel and current categories/products at the six viewport widths.

For a separately authorized release, review/merge then-deployed restrictive rules and publish the prepared indexes before releasing API/UI. Composite indexes cover order filters/aggregate summaries, sorted/category browse and inventory audit pages; emulator success does not verify hosted indexes. Review shipping fee approval and preserve existing approved production settings. If configuration is absent, an approved administrator must explicitly review and save it; do not initialize or overwrite it automatically. Confirm Cloudinary signed-preset permissions for the new `parinaya-silks/home/{device}/...` destinations. No new credentials, online payment provider, notification service or serverless entrypoint is required. Then perform owner acceptance on Vercel. Hosted behavior, index readiness, policy approval and production content remain unverified here.

Validation record

Unit/typecheck/build passed during implementation. The cached Vercel Node builder packaged and invoked the function without `tsx`, using safe configuration-failure probes with `require(ESM)` enabled and disabled. Local API regressions passed for COD concurrency/retry/ownership/transitions and business authorization/media/privacy. Expanded final emulator and browser regression results are recorded after completion below.

Final results

- `npm.cmd run lint`: passed (TypeScript checking).
- `npm.cmd test`: **66 passed, 0 failed**.
- `npm.cmd run build`: passed; Vite retains its existing large-chunk advisory.

- `npm.cmd run test:admin`: **passed as one combined run**: all 10 emulator suites and all 10 browser suites, including the original auth/admin/category/product/shopping/address/appearance/shipping regressions plus COD/business tests.
- COD regressions cover simple and colour-specific final-unit races, duplicate/lost responses, additions after purchase, changed prices/coupons, ownership/tampered fields, exactly-once cancellation and inspected-return restoration, invalid transitions, independent collection, bounded external refunds, deduplicated requests/notifications and legacy profile editing.
- Browser checks cover 320/375/390/768/1024/1440px, including checkout, customer/admin orders, new coupon/content/customer/report screens and existing responsive flows. Inspected mobile and desktop order screenshots; synthetic media only.
- Separate empty-storefront/legacy-cleanup browser checks passed on local development port 3000 and production preview port 4173, including direct refresh, private gates and six widths.
- `npm.cmd run test:catalog-runtime`: passed with cached Vercel Node builder 24.0.3, nodejs24.x artifact, local Node 22.22.2, packaged invocation without the TypeScript loader and safe configuration failures with require(ESM) both enabled/disabled. This validates packaging/import execution, not hosted Vercel configuration or a production database operation.
- `git diff --check`: clean. Generated packaging output was moved into ignored `.cache/completion-vercel-artifact`; no linked Vercel project was moved.

No commits, pushes, deployments, rule/index publication, live-user/product/order/settings changes or real business notifications were performed. No production acceptance was attempted. Rule/index deployment, approved business content/return policy and shipping-fee approval, signed Cloudinary preset acceptance and owner Vercel testing remain future release work. Added rule entries are explicit server-only denials; customer orders/notifications/settings do not require a new browser write permission. The existing shipping revision-listener read must be present in deployed rules for live invalidation. Review deployed rules instead of blindly overwriting them.

Remaining deliberate limits: exports cover the current authorized filtered order page; customer name search is a case-sensitive prefix; substring/colour search and low-stock views use bounded continuation pages; the sitemap covers up to 1,000 products/categories before sharding is needed; metadata is client-rendered; reports use order-creation dates rather than collection-date cashflow. No automatic replacement exchange shipment, refund transfer, courier integration, analytics or online payment is claimed. Customers receive order updates through the website. Razorpay remains on hold.

Reviewable file inventory

- `README.md`
- `docs/COMPLETION.md`
- `firestore.indexes.json`
- `firestore.rules`
- `package.json`
- `public/robots.txt`
- `scripts/admin-browser.test.py`
- `scripts/admin-emulator-suite.ts`
- `scripts/business-browser.test.py`
- `scripts/business-emulator.test.ts`
- `scripts/catalog-handler.test.ts`
- `scripts/cod-browser.test.py`
- `scripts/cod-fixture-control.ts`
- `scripts/customer-browser.test.py`
- `scripts/delivery-browser.test.py`
- `scripts/orders-emulator.test.ts`
- `scripts/shipping-browser.test.py`
- `scripts/storefront-browser.test.py`
- `server/browse.ts`
- `server/business.ts`
- `server/catalog.ts`
- `server/content.ts`
- `server/delivery.ts`
- `server/handler.ts`
- `server/loginAppearance.ts`
- `server/orders.ts`
- `server/seo.ts`
- `src/App.tsx`
- `src/admin/AdminLayout.tsx`

- src/admin/AdminPanel.tsx
- src/admin/ProductManager.tsx
- src/appearance/AdminLoginAppearance.tsx
- src/components/Footer.tsx
- src/content/AdminContent.tsx
- src/content/Seo.tsx
- src/content/model.test.ts
- src/content/model.ts
- src/content/public.tsx
- src/customer/CustomerAccount.tsx
- src/customer/MobileField.tsx
- src/customer/policy.ts
- src/delivery/model.ts
- src/delivery/useCheckoutReview.ts
- src/index.css
- src/orders/AdminBusiness.tsx
- src/orders/InventoryAdjuster.tsx
- src/orders/client.ts
- src/orders/model.test.ts
- src/orders/model.ts
- src/orders/ui.tsx
- src/orders/useCodOrder.ts
- src/products/PublicProduct.tsx
- src/products/client.ts
- src/products/hooks.ts
- src/views/CheckoutView.tsx
- src/views/ContactView.tsx
- src/views/HomeView.tsx
- src/views/PoliciesView.tsx
- src/views/ShopView.tsx
- vercel.json